# Programação 2

## Aula 13 - Orientação a Objetos: herança, tipos abstratos e polimorfismo

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

# Recapitulando

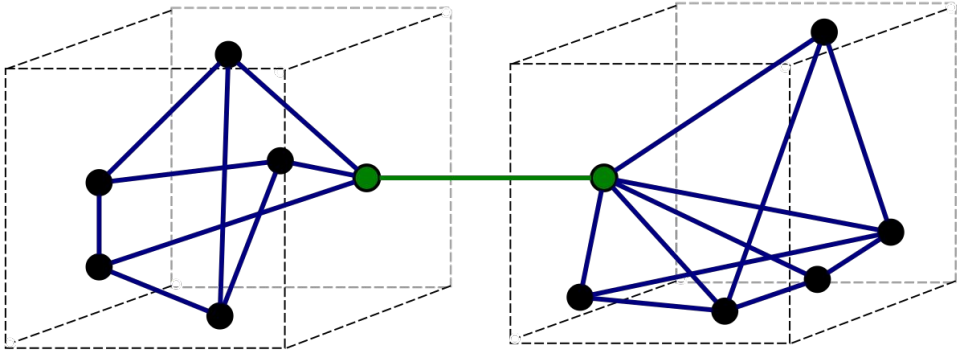
# Coesão e Acoplamento

## Coesão

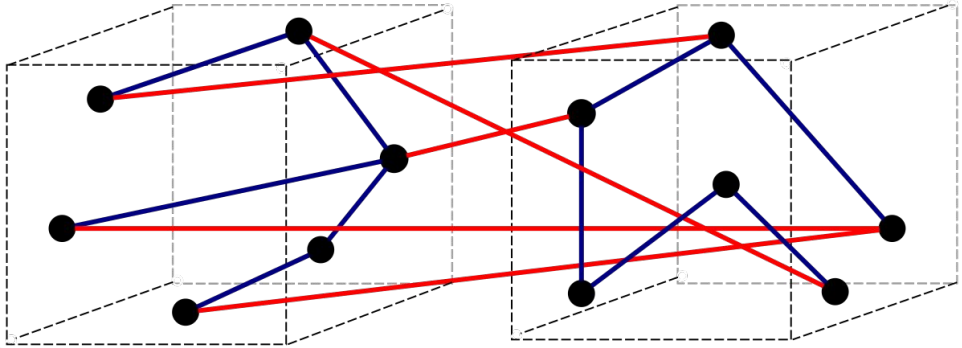
Se refere ao relacionamento que os membros de um módulo possuem. Códigos coesos são aqueles de relação forte, onde seus **membros estão intimamente ligados e estão ali por um objetivo comum**. Membros que não são absolutamente necessários para aquele módulo não devem estar presentes em códigos coesos.

## Acoplamento

Se refere ao relacionamento entre os módulos. Indica **quanto um módulo depende de outro** para funcionar. Códigos desacoplados são aqueles de relação fraca, e não dependem de outros para fazer sua funcionalidade básica. É sempre desejável um baixo nível de acoplamento.



a) Good (loose coupling, high cohesion)



b) Bad (high coupling, low cohesion)

[java - O que são os conceitos de coesão e acoplamento? - Stack Overflow em Português]

# Google Style Guide

## Google Java Style Guide

### Table of Contents


**1 Introduction**
- 1.1 Terminology notes
- 1.2 Guide notes

**2 Source file basics**
- 2.1 File name
- 2.2 File encoding: UTF-8
- 2.3 Special characters

**3 Source file structure**
- 3.1 License or copyright information, if present
- 3.2 Package statement
- 3.3 Import statements

- 4.6 Whitespace
- 4.7 Grouping parentheses: recommended
- 4.8 Specific constructs

**5 Naming**
- 5.1 Rules common to all identifiers
- 5.2 Rules by identifier type
- 5.3 Camel case: defined

**6 Programming Practices**
- 6.1 @Override: always used
- 6.2 Caught exceptions: not ignored
- 6.3 Static members: qualified using class


https://google.github.io/styleguide/javaguide.html

# Conceito de Hoje

## **Herança e Polimorfismo**

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

# Introdução

Problema

Motivação

História e Pessoas

# O que fazer quando

Temos duas classes com várias coisas em comum?
Copiamos uma na outra?
Tem alguma forma de evitar isso?

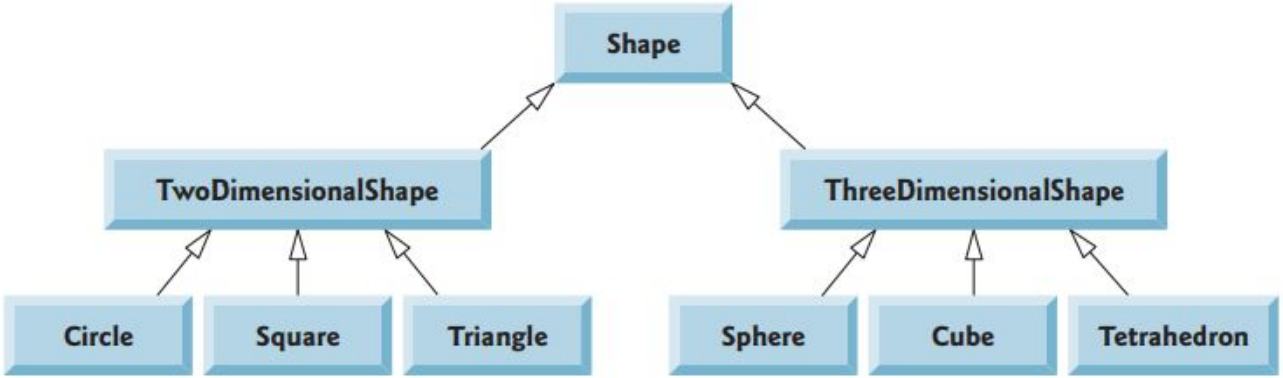
# Apresentação

Definições

Exemplos Ilustrativos

# Herança

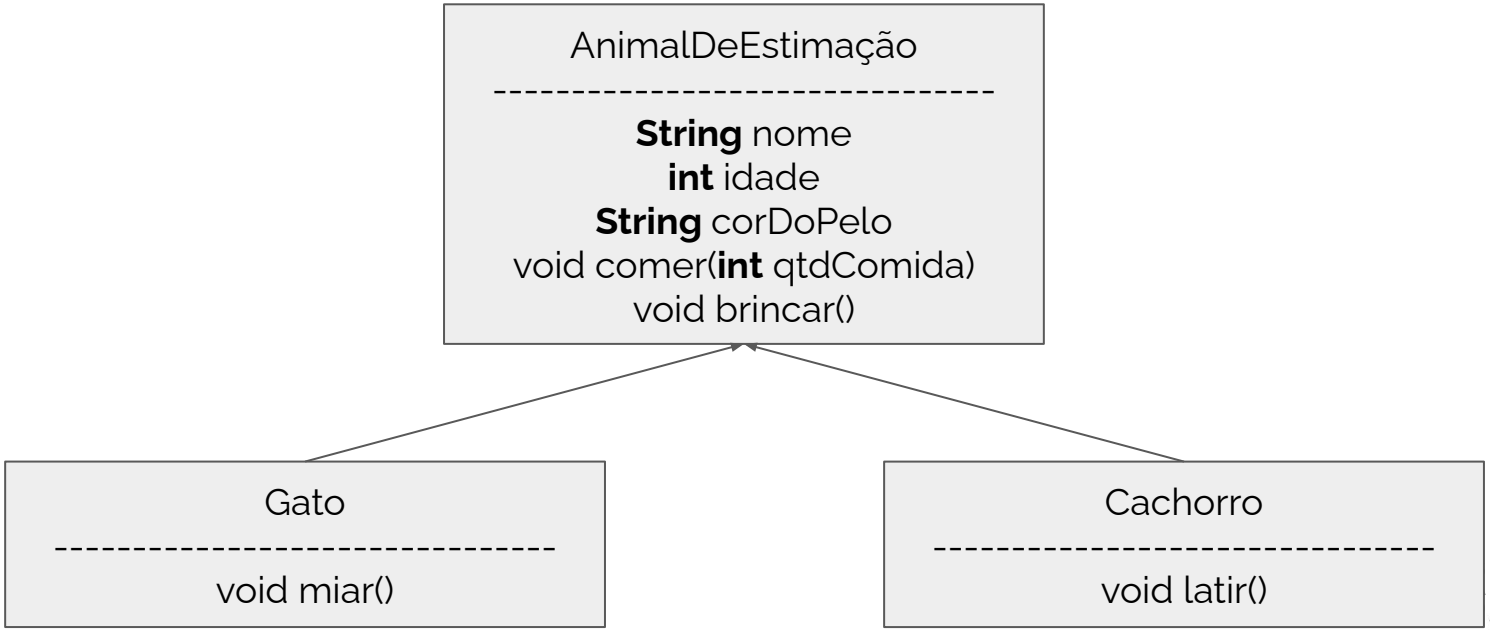
Herança é uma forma de explicitar que uma determinada classe **tem** e **faz** tudo que outra classe faz, e mais um pouco.




***Atenção!*** *Cuidado para não confundir classes com objetos. No exemplo acima temos classes que herdam de outras classes. Para cada uma dessas classes podem existir vários objetos*

# Herança

Herança é uma forma de explicitar que uma determinada classe **tem** e **faz** tudo que outra classe faz, e mais um pouco.

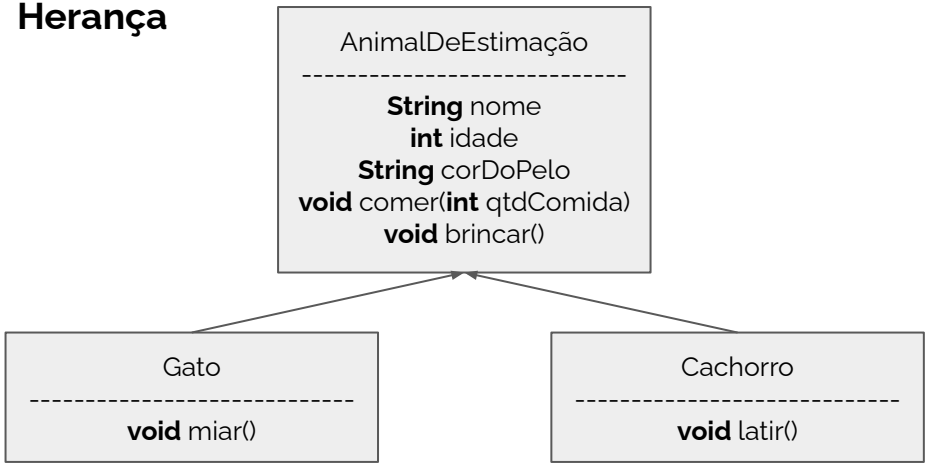



UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

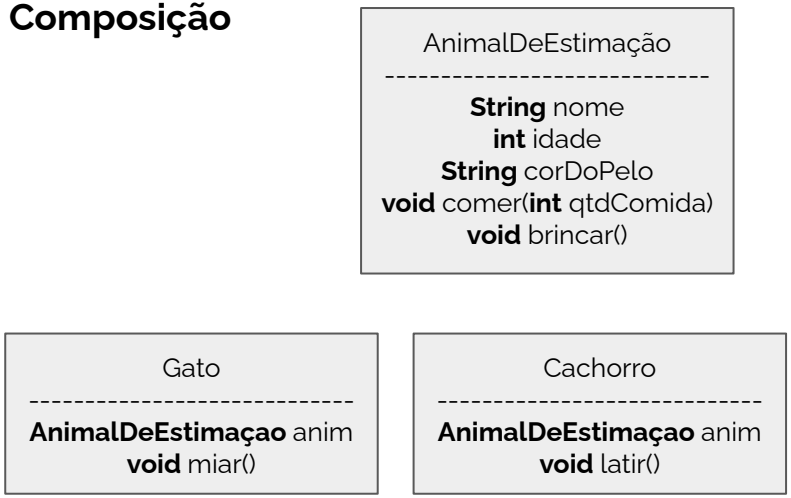
# Herança x Composição

Uma opção para resolver o mesmo tipo de problema é usar a composição

**Herança**




**Composição**



# Herança: SubClasse e SuperClasse

Ao criar uma classe, em vez de declarar membros completamente novos, você pode designar que a nova classe, chamada de **subclasse**,
**deva herdar membros** de uma classe existente, que é chamada de **superclasse**.

*(A linguagem de programação C++ refere-se à superclasse como a classe básica e a subclasse como a classe derivada.) Uma subclasse pode tornar-se uma superclasse para futuras subclasses*

# Herança: SubClasse

Uma subclasse

- pode adicionar seus próprios campos (**atributos**) e **métodos**, portanto, ela é mais específica que sua superclasse
- exibe os comportamentos da superclasse e **pode modificá-los** de modo que eles operem adequadamente para a subclasse
- é por isso que a herança é às vezes chamada **especialização**.

# Herança: Hierarquia

A **superclasse direta** é aquela a partir da qual a subclasse herda explicitamente.

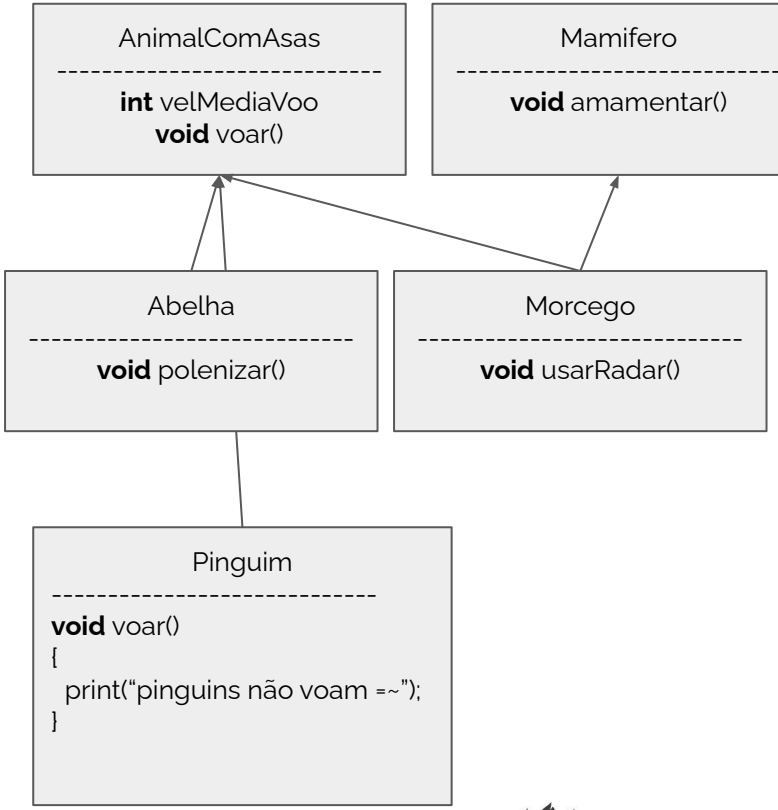
Uma **superclasse indireta** é qualquer classe acima da superclasse direta na **hierarquia de classes**, que define os relacionamentos de herança entre classes

No Java, a hierarquia de classes inicia com a classe **Object**, da qual toda classe em Java direta ou indiretamente **estende** (ou “herda de”)

# Herança: Única x Múltipla

**Herança única** é aquela em que cada classe é derivada **exatamente de uma** superclasse direta

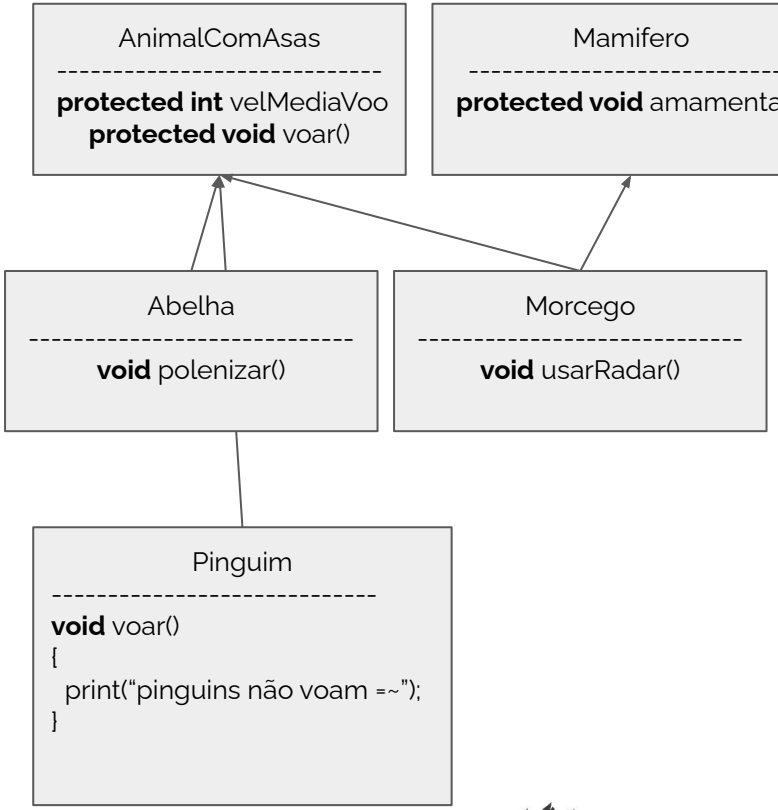
Ao contrário de C++, o Java não suporta **herança múltipla**, que ocorre quando uma classe é derivada de **mais de uma** superclasse direta



# Modificadores de Acesso: Public x Private x Protected

Tudo em uma classe que consta como **private** é inacessível para outras classes, até aquelas que a herdam

Os membros **protected** de uma superclasse podem ser acessados por subclasses



# Herança: Construtores em Subclasses

Instanciar um objeto de uma subclasse inicia uma **cadeia de chamadas do construtor** na qual ele, antes de realizar suas tarefas,

- usa *explicitamente* **super** para chamar um dos construtores em sua superclasse direta,
- ou então para chamar *implicitamente* o **construtor padrão** ou sem argumentos da superclasse.

# Exercício

Inheritance in Java - GeeksforGeeks

```java
class A {
    public void print_A() { System.out.println("Class A"); }
}

class B extends A {
    public void print_B() { System.out.println("Class B"); }
}

class C extends A {
    public void print_C() { System.out.println("Class C"); }
}

class D extends A {
    public void print_D() { System.out.println("Class D"); }
}

// Driver Class
public class Test {
    public static void main(String[] args)
    {
        B obj_B = new B();
        obj_B.print_A();
        obj_B.print_B();

        C obj_C = new C();
        obj_C.print_A();
        obj_C.print_C();

        D obj_D = new D();
        obj_D.print_A();
        obj_D.print_D();
    }
}
```

**Console:**

```
Class A
Class B
Class A
Class C
Class A
Class D
```

# Exercício

```java
class A {
    public void print_() { System.out.println("Class A"); }
}

class B extends A {
    public void print_() { System.out.println("Class B"); }
}

class C extends A {
    public void print_() { System.out.println("Class C"); }
}

class D extends A {
    public void print_() { System.out.println("Class D"); }
}

// Driver Class
public class Test {
    public static void main(String[] args)
    {
        B obj_B = new B();
        obj_B.print_();

        C obj_C = new C();
        obj_C.print_();

        D obj_D = new D();
        obj_D.print_();

    }
}
```

Inheritance in Java - GeeksforGeeks

**Console:**

```
Class B
Class C
Class D
```

# Polimorfismo

O que significa polimorfismo?

Várias formas

# Polimorfismo

O polimorfismo permite

"**programar no geral**"

em vez de

"**programar no específico**"

# Polimorfismo: Exemplos

Se a classe **Retângulo** é derivada da classe **Quadrilátero**, então um objeto Retângulo **é uma versão mais específica** de um objeto Quadrilátero

Qualquer operação (por exemplo, calcular o perímetro ou a área) que pode ser realizada em um objeto Quadrilátero **também pode ser realizada** em um objeto Retângulo

# Polimorfismo: Exemplos

Se a classe **Retângulo** é derivada da classe **Quadrilátero**, então um objeto Retângulo **é uma versão mais específica** de um objeto Quadrilátero

```java
abstract class Forma2D {
    public double area;
    abstract public void calcularArea();
}

class Retangulo extends Forma2D {
    public double altura;
    public double base;

    public Retangulo(double altura, double base)
    {
        this.altura = altura;
        this.base = base;
        this.calcularArea();
    }

    public void calcularArea(){
        this.area = altura * base;
    }

    public String toString(){
        return "Retangulo de altura: " + altura + " e base: " + base;
    }
}
```

```java
class Circulo extends Forma2D {
    public double raio;

    public Circulo(double raio)
    {
        this.raio = raio;
        this.calcularArea();
    }

    public void calcularArea(){
        this.area = Math.PI * Math.pow(raio, b: 2);
    }

    public String toString(){
        return "Criculo de raio: " + raio;
    }
}
```

```java
public static void main(String[] args) {

    Retangulo r1 = new Retangulo(altura: 1, base: 2);
    Retangulo r2 = new Retangulo(altura: 3, base: 5);
    Circulo c1 = new Circulo(raio: 7);
    Circulo c2 = new Circulo(raio: 9);

    ArrayList<Forma2D> formas = new ArrayList<Forma2D>();
    formas.add(r1);
    formas.add(r2);
    formas.add(c1);
    formas.add(c2);

    for (Forma2D forma : formas) {
        System.out.println(forma);
        System.out.println(">> a área dessa forma é: " + forma.area + "\n");
    }

    System.out.println(r1);
    System.out.println(r2);
    System.out.println(c1);
    System.out.println(c2);
```

# Exercício

# Mini-Projeto 8

Parte 1: Ajustes de leitura e construção:

O **carregamento** dos capítulos deve ser feito pela classe Capitulo.

A classe Capitulo deve ter um **novo construtor**, que recebe um objeto do tipo **Scanner como parâmetro**, e executa o método de leitura (antigo método lerCapitulo()) a partir do escaneador fornecido para popular seu objeto.

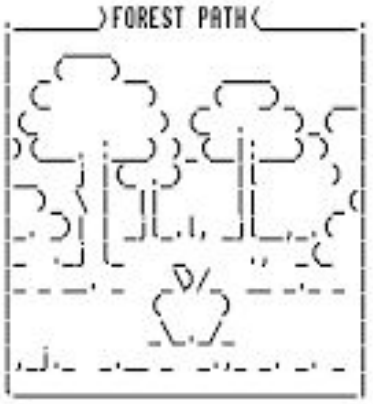
Parte 2: Aplicação da herança e polimorfismo

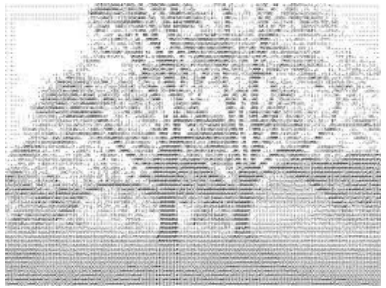
Crie uma **subclasse chamada CapituloImagem**. Esta classe deve ter um atributo a mais **String imagem**, que contém uma imagem no **formato ASCII** (são imagens criadas a partir de sequências de caracteres, como os exemplos ao lado).

O **método mostrar()** desta classe deve mostrar além dos conteúdos anteriores, a imagem do capítulo.

Durante a leitura do arquivo de capítulos, crie “códigos” para indicar se o capítulo terá ou não imagem (ex.: CAPITULO e CAPITULO_COM_IMAGEM). E crie o objeto de acordo com o “código” estabelecido.

Use o construtor da classe passando o Scanner de arquivos como parâmetro



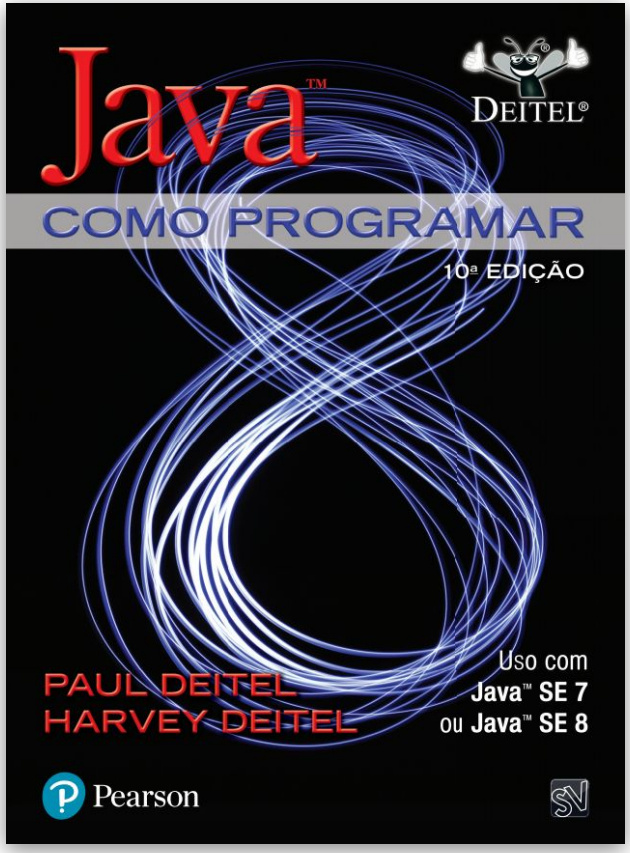
LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

# Referências

## Java - Como Programar (10a Edição)

9. Programação orientada a objetos: herança
    - 9.2 Superclasses e subclasses
    - 9.3 Membros protected
    - 9.5 Construtores em subclasses

10. Programação orientada a objetos: polimorfismo e interfaces
    - 10.4 Classes e métodos abstratos
    - 10.6 Atribuições permitidas entre variáveis ...
    - 10.9 Criando e utilizando interfaces

## Ferramenta:

Eclipse: [eclipse.org/downloads](eclipse.org/downloads)